КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ ІМЕНІ ТАРАСА
ШЕВЧЕНКА
Факультет комп’ютерних наук та
кібернетики

Звіт до етапу 1
лабораторної роботи №1

з дисципліни
«Інформаційні технології»

Студентки четвертого курсу
Групи МІ-41
Факультету комп’ютерних
наук та кібернетики
Шульгіної Анастасії
Максимівни

Київ – 2026

Зміст

1. Аналіз вимог 3
2. Проєктування на високому рівні 5
    2.1. Діаграма прецедентів 5
    2.2. Високорівнева діаграма класів 6
    2.3. Діаграма розгортання 7
3. Деталізоване проєктування 8
    3.1. VOPC — View File Content 8
    3.2. VOPC — Synchronize Local Folder 9
    3.3. Діаграма послідовності — Synchronization 11
    3.4. Діаграма послідовності — Upload File 12
    3.5. Діаграма активності — Synchronization 14
    3.6. Діаграма станів — File 15
    3.7. Діаграма компонентів 16
    3.8. ERD — структура бази даних 17
4. Висновки 18

ЗВІТ

з аналізу вимог та UML-проєктування

системи «Remote File Storage System»

Мета. Аналіз вимог і проєктування структури та поведінки системи віддаленого зберігання файлів.

Backend та storage. FastAPI backend; Render; Supabase Auth, PostgreSQL і Storage.

1. Аналіз вимог

Система є спрощеним віддаленим файловим сховищем, у якому кожен користувач працює зі своїм віртуальним диском. Користувач проходить автентифікацію, переглядає файли та метадані, виконує файлові операції й може вручну запускати синхронізацію локальної папки з віддаленою папкою.

- реєстрація, вхід і вихід із системи;
- перегляд віртуального диска та віддаленого сховища;
- перегляд метаданих файлів: ім'я, розмір, дати створення/зміни, користувач завантаження та останньої зміни;
- завантаження окремого файлу, включно з drag-and-drop як додатковим способом;
- завантаження файлу з віддаленого сховища та його збереження локально;
- видалення файлу з підтвердженням;
- preview .cpp як тексту та .png як зображення; для інших типів preview недоступний;
- сортування за датою/часом створення у висхідному або низхідному порядку;
- фільтрація за .c та .jpg або перегляд усіх файлів;
- налаштування видимості додаткових колонок; Name завжди видима;
- ручна синхронізація через кнопку Synchronize із вибором локальної та віддаленої папки;
- порівняння файлів і автоматичне визначення змін;
- при конфлікті зберігається версія з пізнішим modifiedAt.

Ключове архітектурне обмеження синхронізації: FastAPI не має доступу до локальної файлової системи. Desktop Client сканує локальну папку, передає manifest серверу, отримує план синхронізації та сам виконує локальні upload/download/delete операції.

2. Проєктування на високому рівні

2.1. Діаграма прецедентів

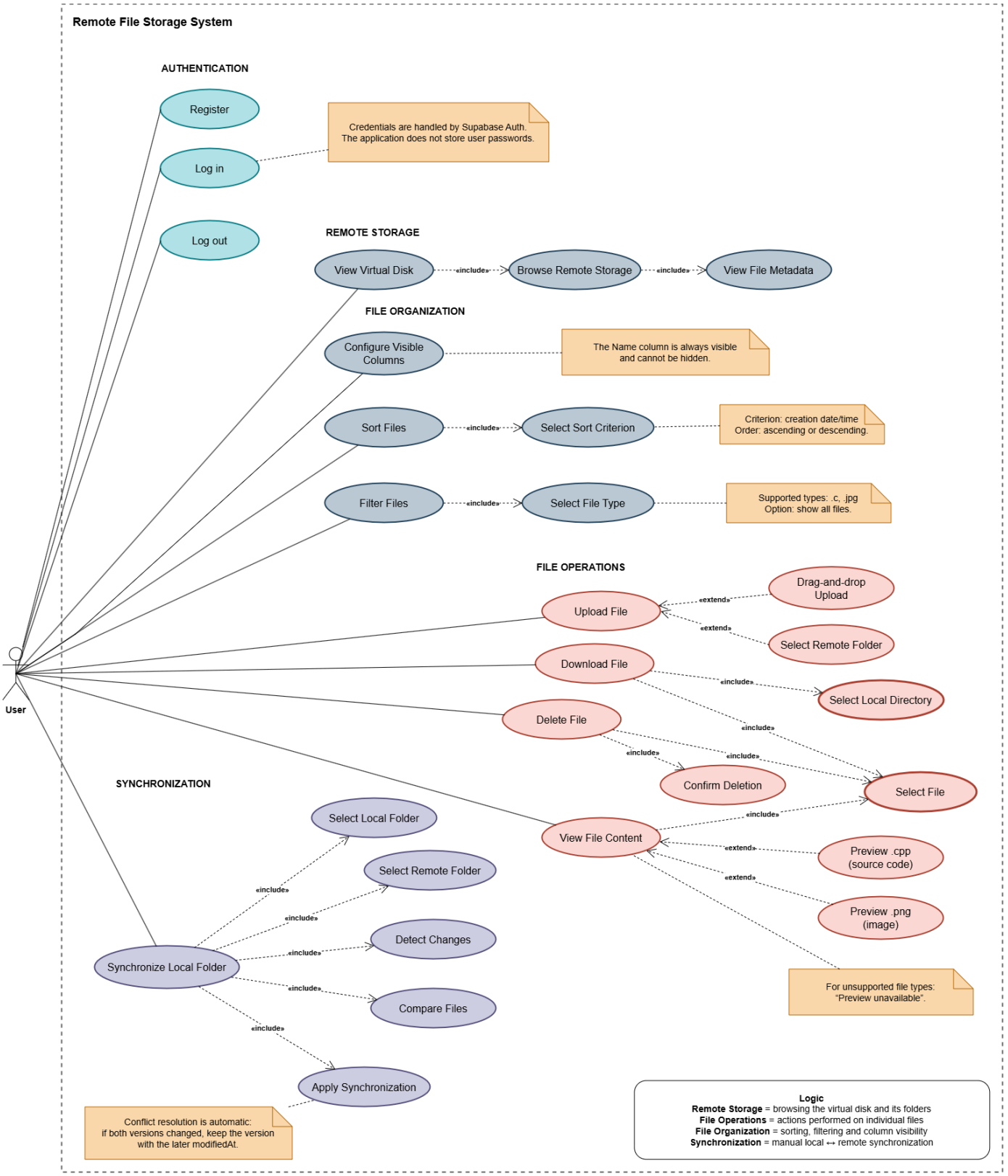


Рисунок 1 — Діаграма прецедентів системи.

Діаграма показує систему з точки зору єдиного актора User. Функції згруповані за автентифікацією, віддаленим сховищем, організацією файлів, файловими операціями та синхронізацією. <<include>> використано для обов'язкових піддій, а drag-and-drop показано як <<extend>> для Upload File.

2.2. Високорівнева діаграма класів

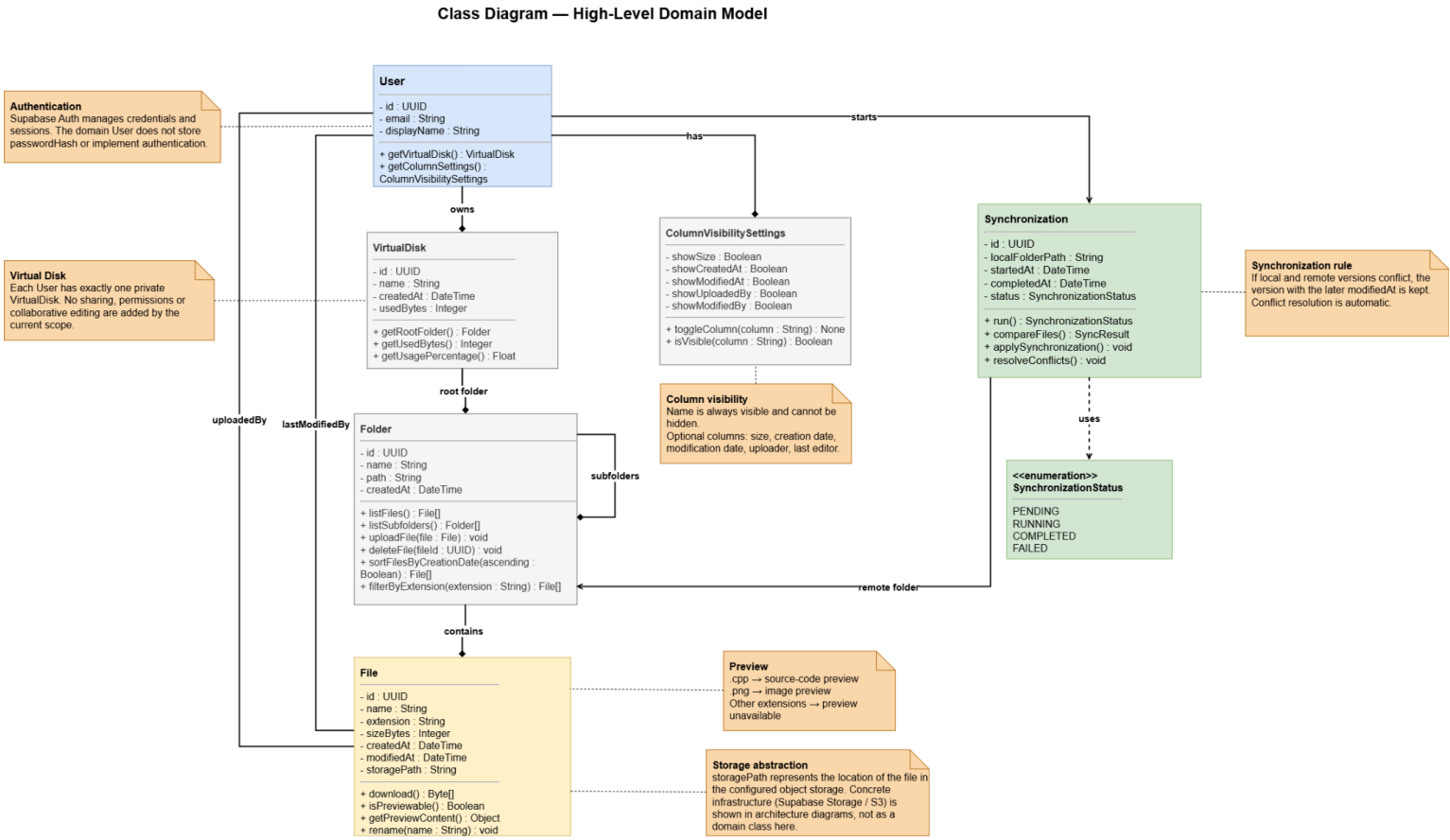


Рисунок 2 — Високорівнева діаграма класів.

Модель містить User, VirtualDisk, Folder, File, Synchronization, ColumnVisibilitySettings та SynchronizationStatus. User володіє одним VirtualDisk; VirtualDisk має root folder; Folder містить підпапки та File. Для File окремо моделюються uploadedBy та lastModifiedBy. Пароль не входить до доменної моделі, оскільки автентифікація винесена до Supabase Auth.

2.3. Діаграма розгортання

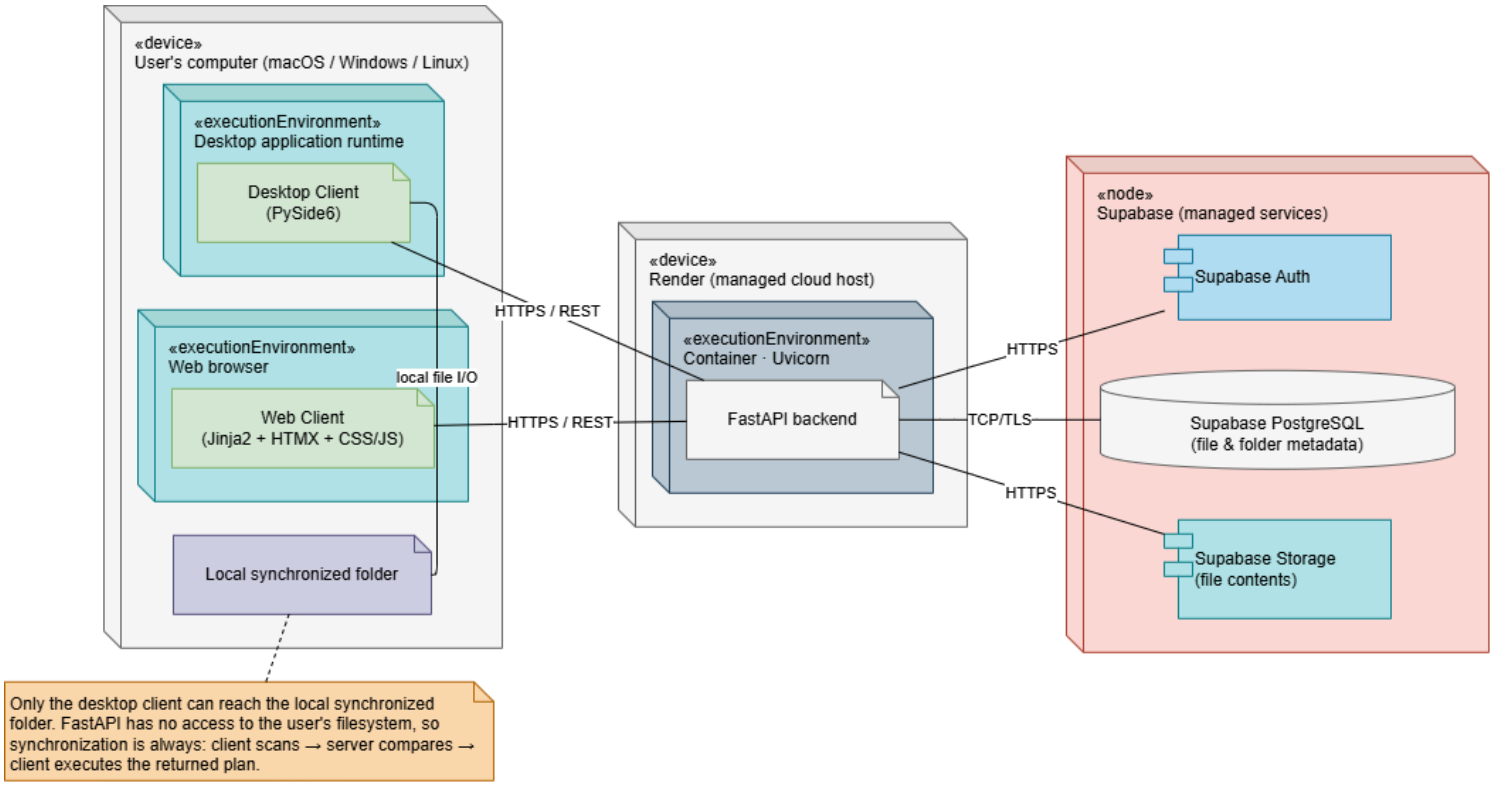


Рисунок 3 — Діаграма розгортання.

На комп'ютері користувача працюють desktop-клієнт і web-клієнт. FastAPI backend розгорнутий у контейнерному середовищі Render. Supabase надає Auth, PostgreSQL для метаданих і Storage для вмісту файлів. Клієнти взаємодіють із backend через HTTPS/REST, а desktop-клієнт додатково має локальний file I/O.

3. Деталізоване проєктування

3.1. VOPC — View File Content

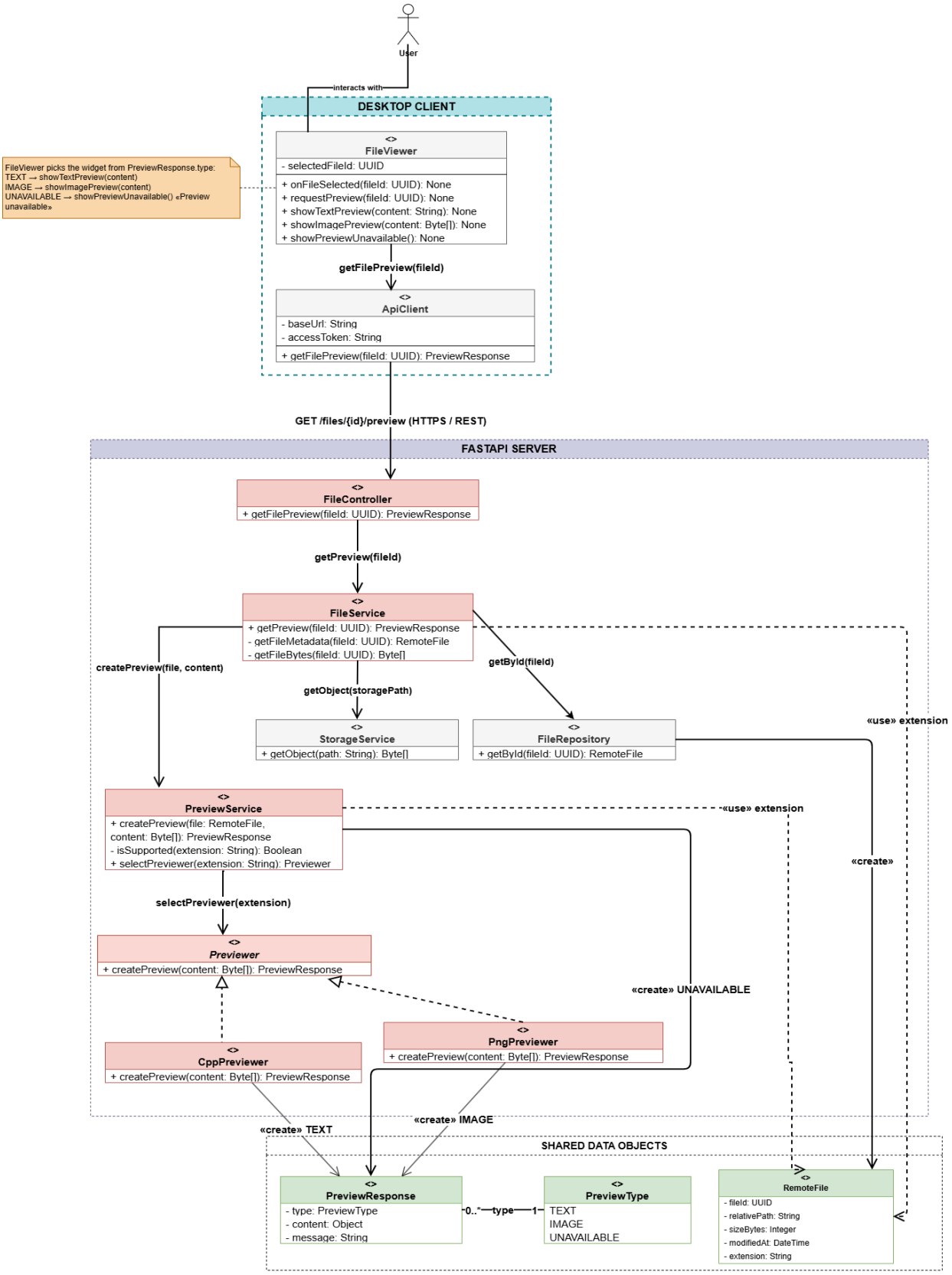


Рисунок 4 — VOPC-діаграма для перегляду вмісту файлу.

3.2. VOPC — Synchronize Local Folder

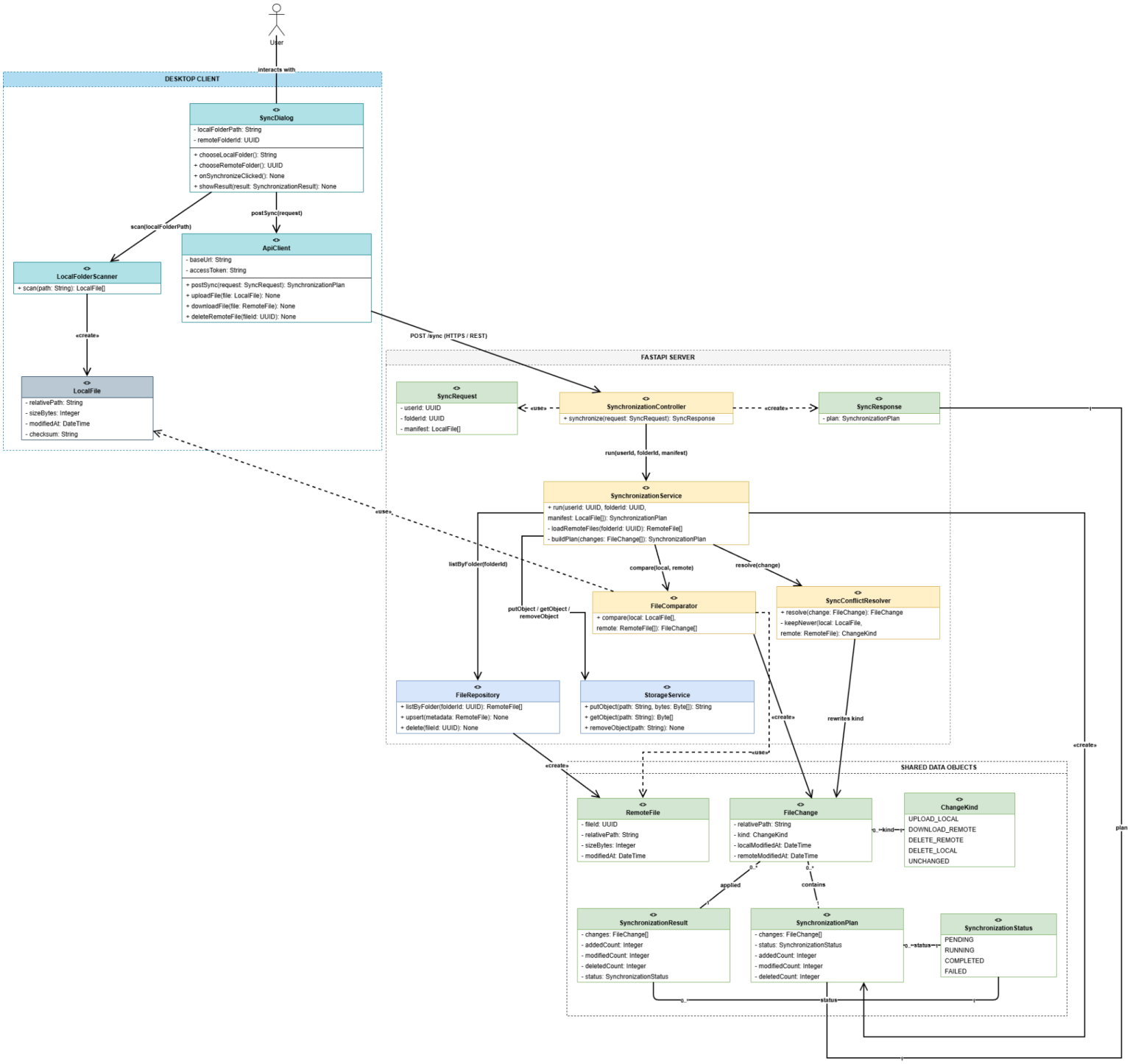


Рисунок 5 — VOPC-діаграма для синхронізації локальної папки.

SyncDialog отримує від користувача localFolderPath і remoteFolderId. LocalFolderScanner створює LocalFile та manifest. ApiClient передає manifest через POST /sync. На сервері SynchronizationController передає дані SynchronizationService, який отримує RemoteFile через FileRepository,

порівнює локальні та віддалені дані через FileComparator і застосовує SyncConflictResolver.

ChangeKind визначає UPLOAD_LOCAL, DOWNLOAD_REMOTE, DELETE_REMOTE, DELETE_LOCAL або UNCHANGED. SynchronizationPlan описує необхідні дії, а SynchronizationResult - результат їх виконання.

3.3. Діаграма послідовності — Synchronization

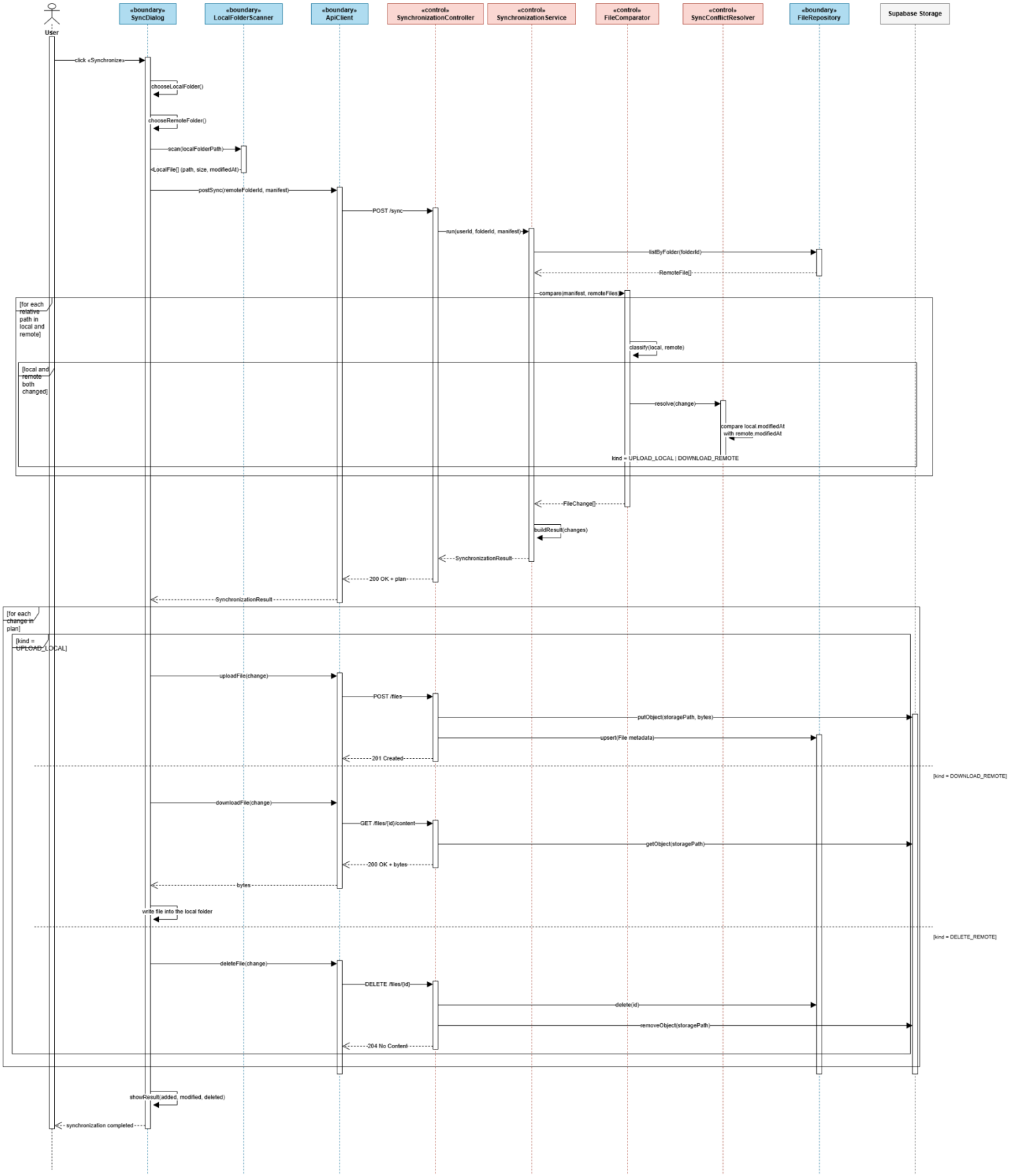


Рисунок 6 — Діаграма послідовності для синхронізації.

Після натискання Synchronize користувач вибирає папки. Desktop Client сканує локальний каталог і формує manifest. Backend отримує remote metadata, порівнює файли за relative path, визначає зміни та для конфліктів порівнює modifiedAt. Сервер повертає план, після чого клієнт виконує відповідні операції й показує результат.

3.4. Діаграма послідовності — Upload File

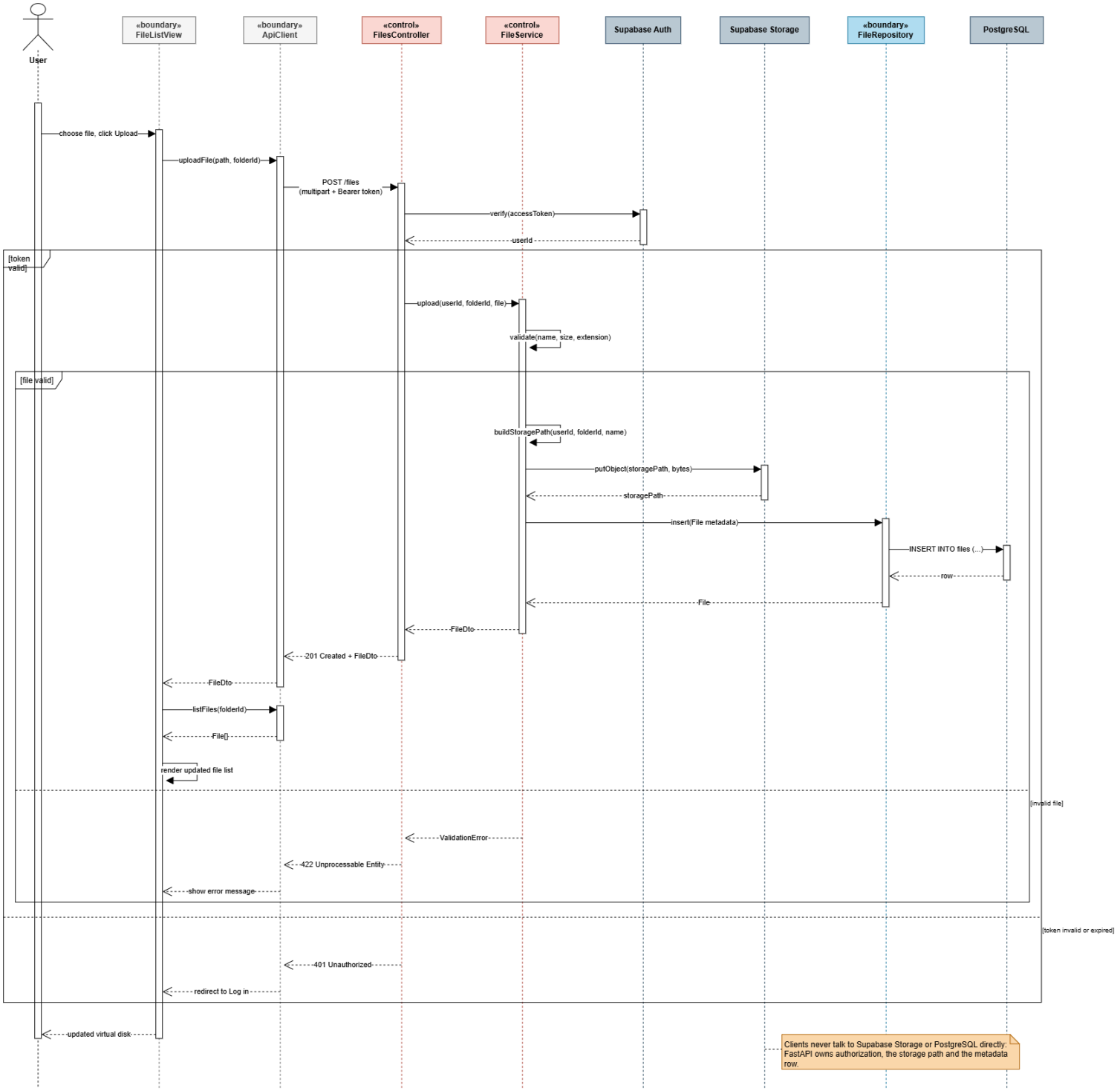


Рисунок 7 — Діаграма послідовності для завантаження файлу.

Користувач вибирає файл. ApiClient надсилає POST /files. FilesController перевіряє access token через Supabase Auth і передає операцію FileService. FileService перевіряє name, size та extension, формує storage path, завантажує bytes у Supabase Storage і створює metadata row через FileRepository/PostgreSQL. Після успіху клієнт отримує FileDto та оновлює список.

На діаграмі також показані негативні сценарії: ValidationError з HTTP 422 та недійсний/прострочений token з HTTP 401.

3.5. Діаграма активності — Synchronization

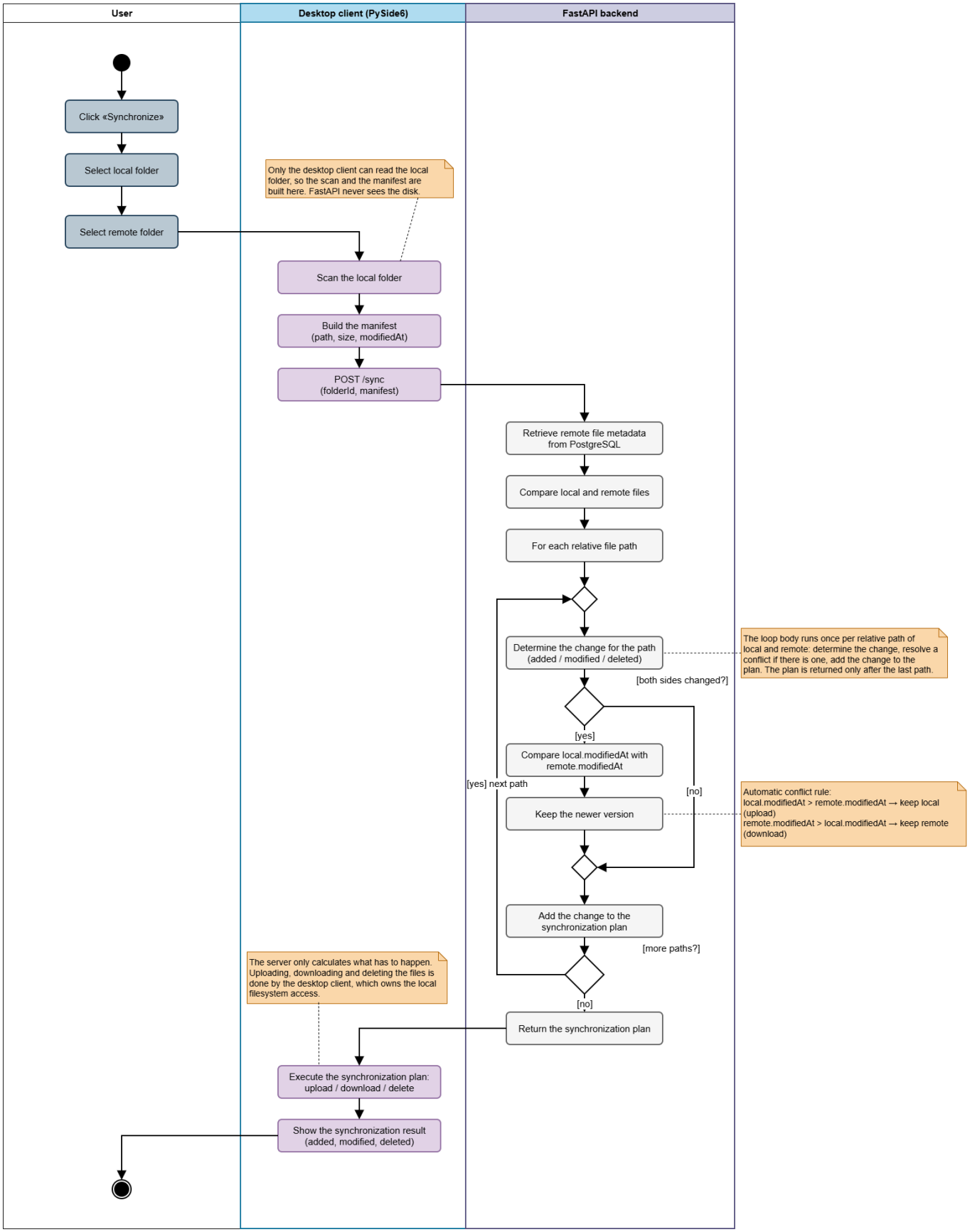


Рисунок 8 — Діаграма активності синхронізації.

Діаграма деталізує алгоритм синхронізації та розподіляє відповідальність між User, Desktop Client і FastAPI backend.

- User запускає Synchronize та вибирає local і remote folder.
- Desktop Client сканує локальну папку і будує manifest.
- Backend отримує remote metadata та порівнює її з manifest.
- Для кожного relative path визначається тип зміни.
- Якщо змінені обидві версії, порівнюються modifiedAt і вибирається новіша.
- Backend повертає synchronization plan.
- Desktop Client виконує upload/download/delete, оскільки має доступ до локальної файлової системи.
- Користувач отримує synchronization result.

3.6. Діаграма станів — File

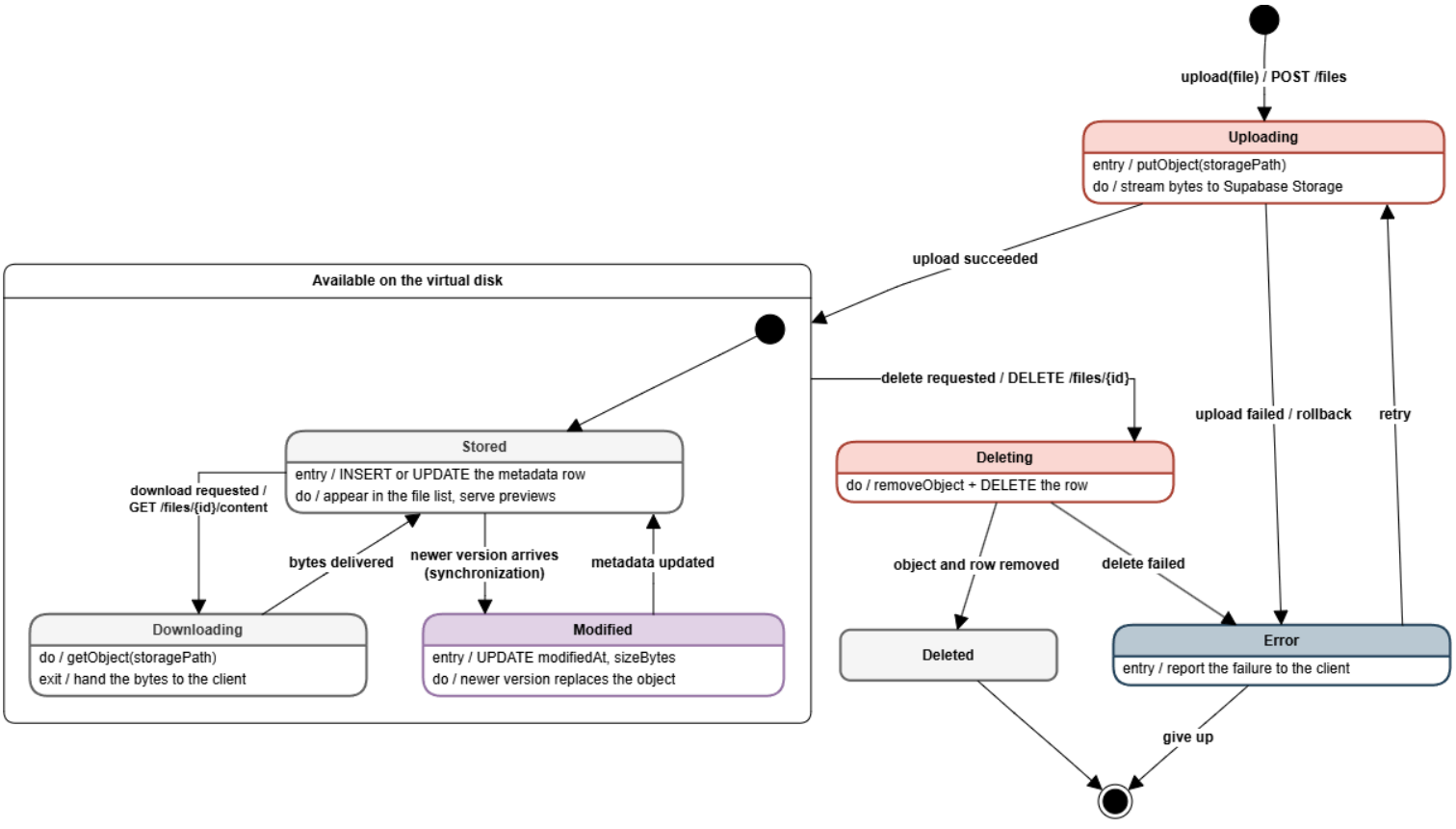


Рисунок 9 — Діаграма станів файлу.

Діаграма описує стани операцій над файлом: Uploading, Stored, Downloading, Modified, Deleting, Deleted та Error. Після успішного upload

файл переходить у Stored. Downloading повертає bytes клієнту, Modified використовується для надходження новішої версії під час synchronization, а Deleting завершується станом Deleted. Для помилки upload передбачено retry.

3.7. Діаграма компонентів

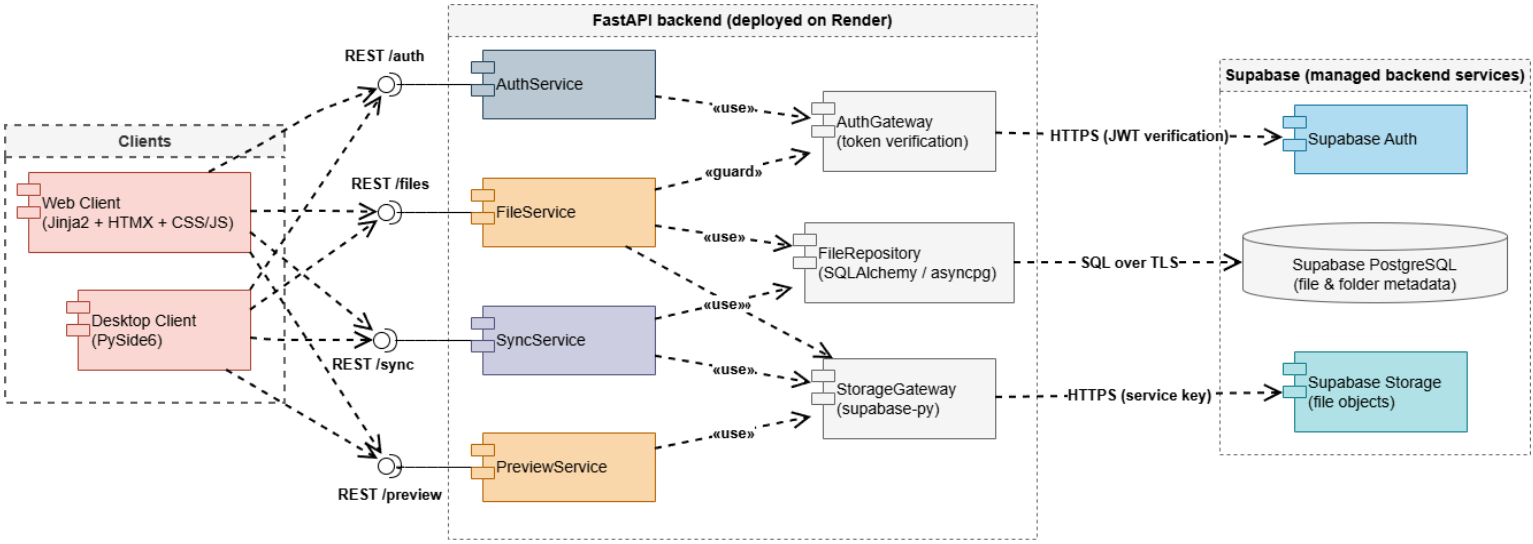


Рисунок 10 — Діаграма компонентів.

Архітектура складається з Web Client і Desktop Client, які звертаються до REST API FastAPI backend. У backend виділено AuthService, FileService, SyncService і PreviewService. Infrastructure-компоненти включають AuthGateway, FileRepository та StorageGateway. FileRepository працює з Supabase PostgreSQL, StorageGateway - із Supabase Storage, а AuthGateway - із Supabase Auth.

Клієнти не звертаються безпосередньо до PostgreSQL або Storage; усі операції з віддаленими даними проходять через FastAPI.

3.8. ERD — структура бази даних

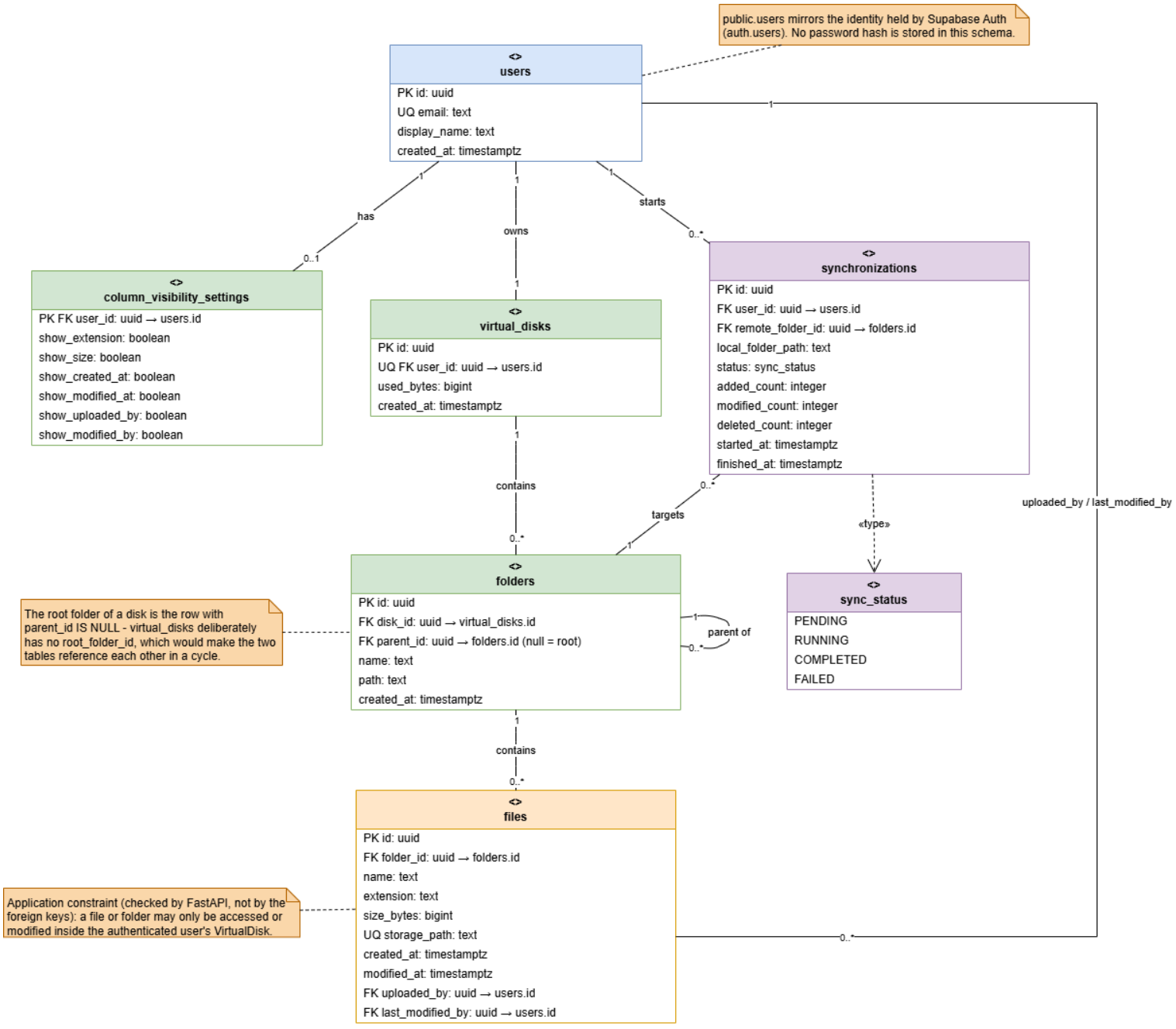


Рисунок 11 — ERD структури бази даних.

ERD конкретизує збереження метаданих у PostgreSQL. Таблиця users містить ідентичність користувача без password hash. virtual_disks пов'язана з users у співвідношенні 1:1. folders належать virtual disk та утворюють ієрархію через parent_id; root folder визначається як запис із parent_id IS NULL. files належать folders і містять storage_path, а uploaded_by та last_modified_by посилаються на users.

column_visibility_settings зберігає налаштування видимості додаткових колонок. synchronizations зберігає статус, часові мітки та статистику доданих, змінених і видалених файлів. Доступ до файлів і папок контролюється FastAPI в межах VirtualDisk автентифікованого користувача.

4. Висновки

У роботі виконано аналіз вимог і послідовне проєктування системи від загального до детального рівня. Use Case Diagram визначає функції системи; High-Level Class Diagram описує доменні сутності; Deployment Diagram показує фізичне розміщення; VOPC, Sequence, Activity та State diagrams деталізують поведінку окремих сценаріїв; Component Diagram описує програмну архітектуру; ERD уточнює структуру PostgreSQL.

Сформована модель є основою для подальшої реалізації та тестування. Особливістю системи є розподіл відповідальності при синхронізації: локальна файлова система доступна desktop-клієнту, сервер аналізує metadata та формує план, а клієнт виконує фактичні локальні операції.